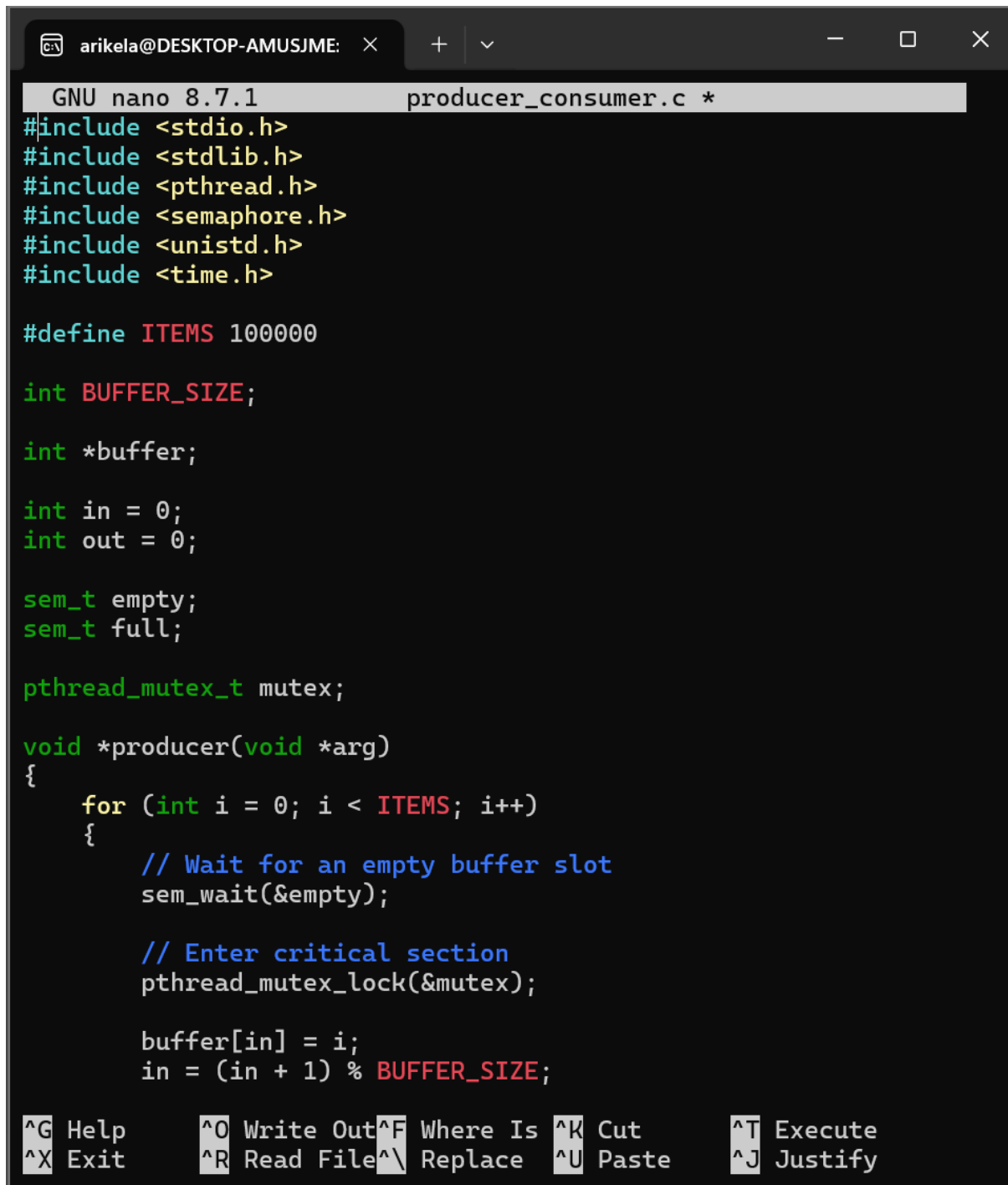


Practical_Week-10

TASK-1: Implement the Producer-Consumer problem using counting semaphores and POSIX threads. Evaluate synchronization correctness and System throughput under different buffer sizes.

ANS:



```
GNU nano 8.7.1 producer_consumer.c *
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <time.h>

#define ITEMS 100000

int BUFFER_SIZE;

int *buffer;

int in = 0;
int out = 0;

sem_t empty;
sem_t full;

pthread_mutex_t mutex;

void *producer(void *arg)
{
    for (int i = 0; i < ITEMS; i++)
    {
        // Wait for an empty buffer slot
        sem_wait(&empty);

        // Enter critical section
        pthread_mutex_lock(&mutex);

        buffer[in] = i;
        in = (in + 1) % BUFFER_SIZE;
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^N Replace	^U Paste	^J Justify

```
    pthread_mutex_unlock(&mutex);

    // Signal that one item is available
    sem_post(&full);
}

return NULL;
}

void *consumer(void *arg)
{
    long long sum = 0;

    for (int i = 0; i < ITEMS; i++)
    {
        // Wait for an available item
        sem_wait(&full);

        // Enter critical section
        pthread_mutex_lock(&mutex);

        int item = buffer[out];
        out = (out + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);

        // Signal that one buffer slot is empty
        sem_post(&empty);

        sum += item;
    }
}
```

^G Help
^X Exit

^O Write Out
^R Read File

^F Where Is
^ Replace

^K Cut
^U Paste

^T Execute
^J Justify

arikela@DESKTOP-AMUSJME: ×

+ v

—

□

×

GNU nano 8.7.1

producer_consumer.c *

```
// Signal that one buffer slot is empty
sem_post(&empty);

    sum += item;
}

printf("Consumer received all %d items.\n", ITEMS);
printf("Consumer sum = %lld\n", sum);

return NULL;
}

int main()
{
    pthread_t producer_thread;
    pthread_t consumer_thread;

    printf("Enter buffer size: ");
    scanf("%d", &BUFFER_SIZE);

    if (BUFFER_SIZE <= 0)
    {
        printf("Invalid buffer size.\n");
        return 1;
    }

    buffer = malloc(BUFFER_SIZE * sizeof(int));

    if (buffer == NULL)
    {
        perror("malloc");
        return 1;
    }
}
```

^G Help

^X Exit

^O Write Out

^R Read File

^F Where Is

^ Replace

^K Cut

^U Paste

^T Execute

^J Justify

```

if (buffer == NULL)
{
    perror("malloc");
    return 1;
}

// Counting semaphores
// Initially all buffer slots are empty
sem_init(&empty, 0, BUFFER_SIZE);

// Initially no items are available
sem_init(&full, 0, 0);

pthread_mutex_init(&mutex, NULL);

clock_t start = clock();

pthread_create(&producer_thread, NULL, producer, NULL);
pthread_create(&consumer_thread, NULL, consumer, NULL);

pthread_join(producer_thread, NULL);
pthread_join(consumer_thread, NULL);

clock_t end = clock();

double elapsed =
    (double)(end - start) / CLOCKS_PER_SEC;

printf("Buffer size = %d\n", BUFFER_SIZE);
printf("Execution time = %.6f seconds\n", elapsed);

if (elapsed > 0)

```

[^]G Help
[^]X Exit

[^]O Write Out
[^]R Read File

[^]F Where Is
[^]\ Replace

[^]K Cut
[^]U Paste

[^]T Execute
[^]J Justify

```

    if (elapsed > 0)
    {
        printf("Throughput = %.2f items/second\n",
               ITEMS / elapsed);
    }

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    free(buffer);

    return 0;
}

```

^{^G} Help ^{^O} Write Out ^{^F} Where Is ^{^K} Cut ^{^T} Execute
^{^X} Exit ^{^R} Read File ^{^N} Replace ^{^U} Paste ^{^J} Justify

```

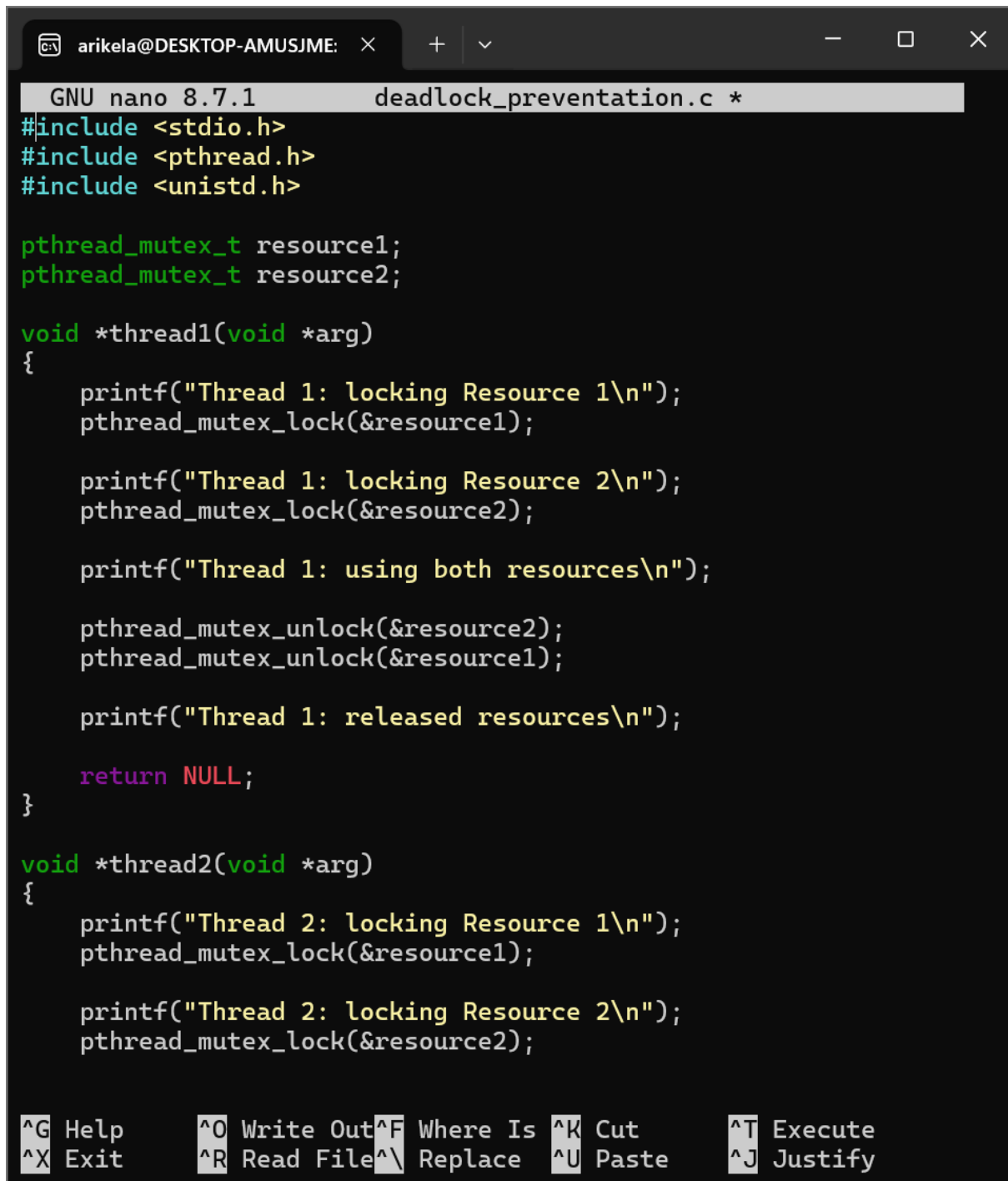
arikel@DESKTOP-AMUSJME:~/week5prac$ ./producer_consumer
Enter buffer size: 5
Consumer received all 100000 items.
Consumer sum = 4999950000
Buffer size = 5
Execution time = 0.666434 seconds
Throughput = 150052.37 items/second
arikel@DESKTOP-AMUSJME:~/week5prac$ ./producer_consumer
Enter buffer size: 100
Consumer received all 100000 items.
Consumer sum = 4999950000
Buffer size = 100
Execution time = 0.066317 seconds
Throughput = 1507908.98 items/second
arikel@DESKTOP-AMUSJME:~/week5prac$ ./producer_consumer
Enter buffer size: 1000
Consumer received all 100000 items.
Consumer sum = 4999950000
Buffer size = 1000
Execution time = 0.061705 seconds
Throughput = 1620614.21 items/second
arikel@DESKTOP-AMUSJME:~/week5prac$

```

TASK-2: Design a deadlock scenario involving multiple threads competing for shared resources. Demonstrate deadlock occurrence, identify the four necessary conditions for deadlock, and implement a deadlock prevention strategy using resource ordering.

ANS:

deadlock_prevention.c:



```
GNU nano 8.7.1 deadlock_prevention.c *
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1;
pthread_mutex_t resource2;

void *thread1(void *arg)
{
    printf("Thread 1: locking Resource 1\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 1: locking Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 1: using both resources\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    printf("Thread 1: released resources\n");

    return NULL;
}

void *thread2(void *arg)
{
    printf("Thread 2: locking Resource 1\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 2: locking Resource 2\n");
    pthread_mutex_lock(&resource2);

    return NULL;
}
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify

```

    printf("Thread 2: using both resources\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    printf("Thread 2: released resources\n");

    return NULL;
}

int main()
{
    pthread_t t1, t2;

    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&resource1);
    pthread_mutex_destroy(&resource2);

    printf("No deadlock occurred.\n");

    return 0;
}

```

```

arikel@DESKTOP-AMUSJME:~/week5prac$ gcc deadlock_prevention.c
-o deadlock_prevention -pthread
arikel@DESKTOP-AMUSJME:~/week5prac$ ./deadlock_prevention
Thread 1: locking Resource 1
Thread 1: locking Resource 2
Thread 1: using both resources
Thread 1: released resources
Thread 2: locking Resource 1
Thread 2: locking Resource 2
Thread 2: using both resources
Thread 2: released resources
No deadlock occurred.
arikel@DESKTOP-AMUSJME:~/week5prac$

```